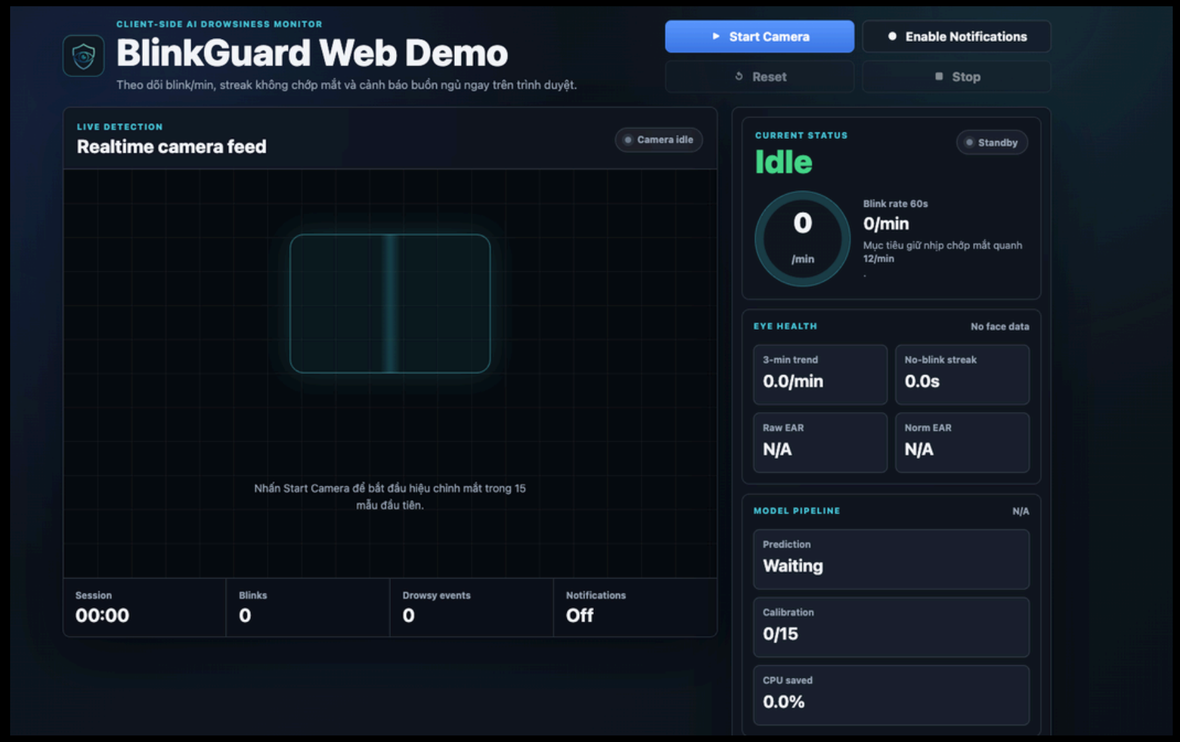
# Deployment Goals

- Deploy the eye-state detection pipeline as a real-time web application.
- Run all inference directly in the user's browser.
- Track blink behavior and drowsiness using webcam input.
- Provide alerts when users forget to blink during long focus sessions.
- Keep the system privacy-friendly by avoiding server-side video processing.



# Why Client-Side Deployment?

## Why Run Everything in the Browser?

- The final TimeSeries EAR model is a Linear SVM.
- Inference is fast enough to run directly on laptops and phones.
- No backend server is required for camera processing or model inference.
- Webcam frames stay on the user's device.
- The app can be hosted as a static website with very low deployment cost.

> Client-side deployment gives us low latency, better privacy, and almost zero server cost.

# Deployed Model

## Model Used for Web Deployment

### Source & Type

- **Source model:** best_traditional_model.pkl
- **Model type:** StandardScaler + Linear-kernel multiclass SVM
- **Input:** 12 dynamic EAR-based features

### Output Classes

- **0:** Eye open / no blink
- **1:** Blink
- **2:** Sleep / long closure

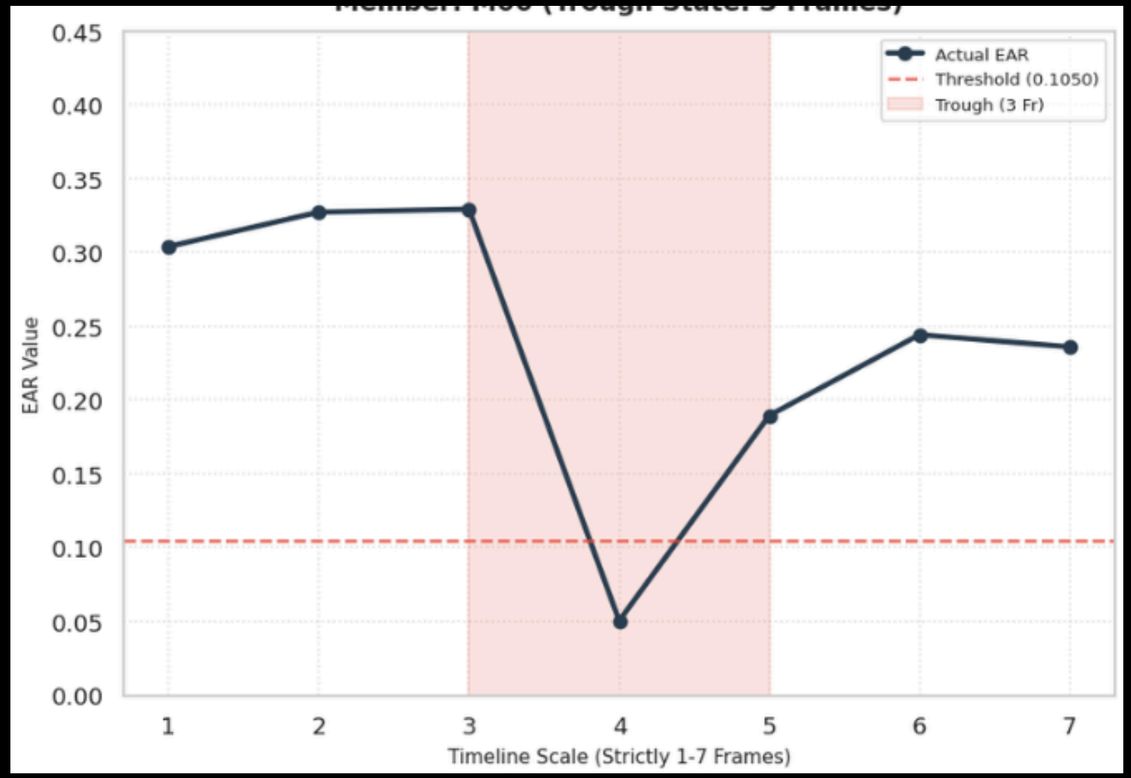
# 12-Feature Input Vector

## Input Features for SVM Inference

- 7 normalized EAR values from a sliding window
- Minimum EAR in the window
- Maximum EAR in the window
- Standard deviation
- Center ratio:

$$\frac{w[0] + w[6]}{2 \cdot w[3] + 1e\text{-}5}$$

- Kurtosis of the window




**Important:**
The JavaScript feature extraction must match the Python training pipeline exactly.

# Converting the Model to JavaScript

From `.pkl` Model to JavaScript Function

Output File:
deployment/scripts/svm_model.js

## Step 1
Load Model

Load the trained model with joblib.

## Step 2
StandardScaler

Extract StandardScaler parameters:
– mean_
– scale_

## Step 3
Linear SVM

Extract Linear SVM parameters:
– coef_
– intercept_
– classes_

## Step 4
Generate Code

Generate pure JavaScript inference code.

# JavaScript Model Runtime

## Pure JavaScript SVM Inference

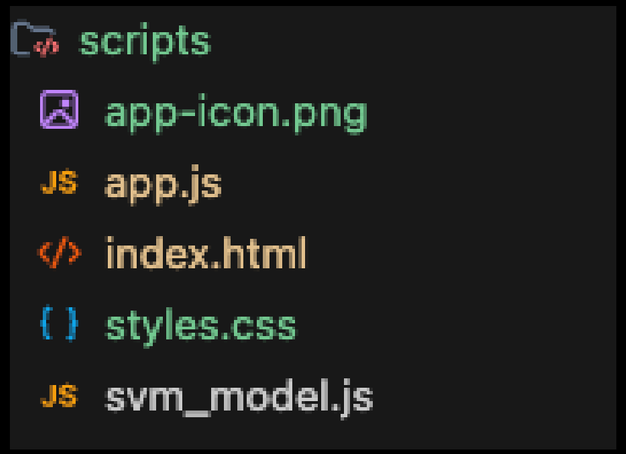
- **scaleFeatures(features)** reproduces StandardScaler.
- **scoreSVM(scaledFeatures)** reproduces Linear SVM scoring.
- **predictPipeline(features)** returns the final class.
- No machine learning library is required in the browser.

CODE CONCEPT:

```
const prediction =
predictPipeline(features);
```

# Deployment Folder Structure

## Static Web App Structure

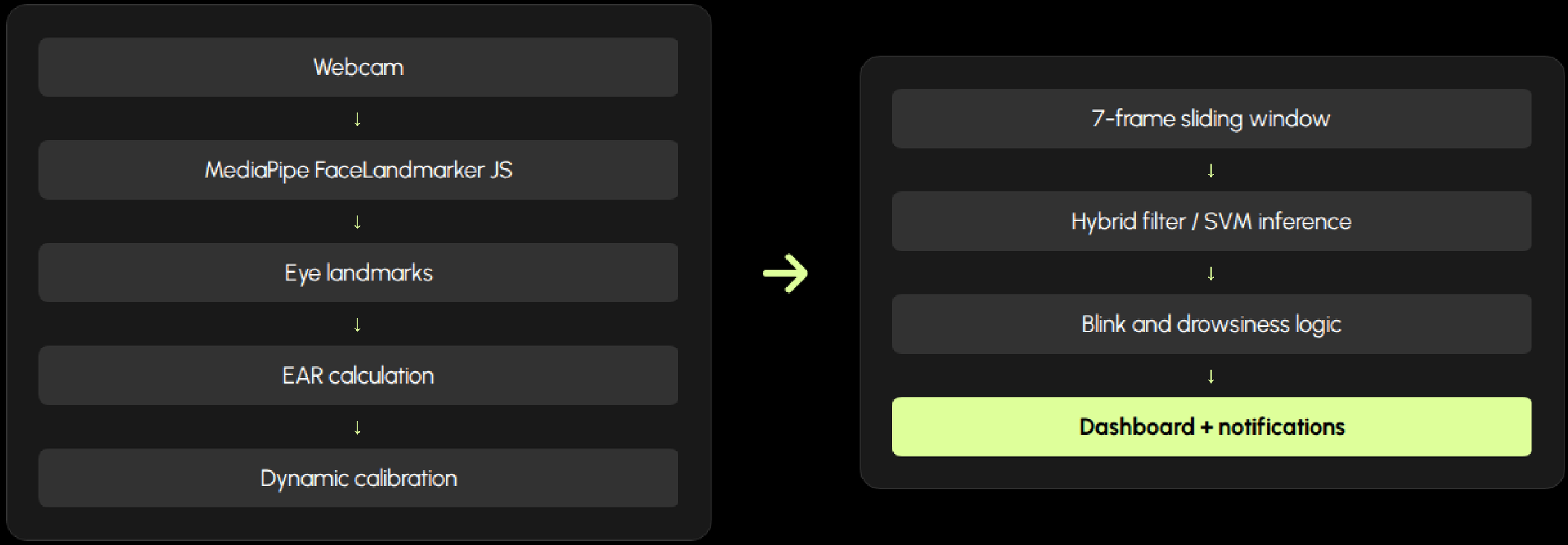



- **index.html:** UI layout, camera view, dashboard, controls
- **app.js:** webcam, MediaPipe, EAR, calibration, SVM, alerts
- **svm_model.js:** converted Linear SVM model
- **app-icon.png:** BlinkGuard shield-and-eye app icon
- **styles.css:** app decoration

> Key Message:
> The entire app can be deployed as static files.

# Runtime Architecture

## Real-Time Browser Pipeline

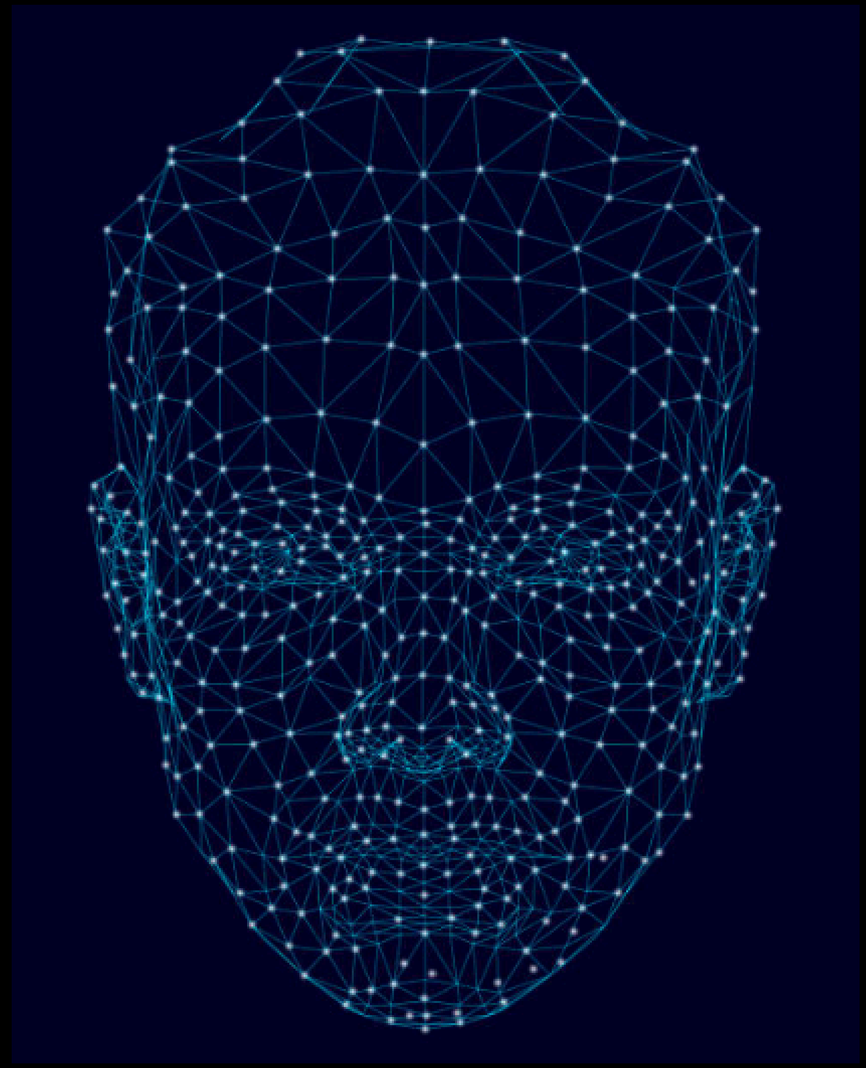


# Eye Tracking with MediaPipe JS

## Browser-Based Face and Eye Tracking

- Uses @mediapipe/tasks-vision.
- Runs FaceLandmarker in video mode.
- Extracts facial landmarks directly from the webcam stream.
- No video frame is uploaded to a server.
- MediaPipe assets are loaded from CDN for MVP simplicity.

> Production Note:
> MediaPipe WASM/model files can be self-hosted later to reduce CDN dependency.



# EAR Calculation in the Browser

## Eye Aspect Ratio on Web Landmarks

### Landmarks

- **Left eye landmarks:**
  [33, 160, 158, 133, 153, 144]
- **Right eye landmarks:**
  [362, 385, 387, 263, 373, 380]

### Formulas

**EAR formula:**

```
EAR = (||p2 - p6|| + ||p3 - p5||) / (2 * ||p1 - p4||)
```

**Final raw EAR:**

```
rawEAR = Math.min(leftEAR, rightEAR);
```

Reason:

Using the smaller EAR increases sensitivity to unilateral eye closure, but may be more susceptible to landmark errors or partial occlusion.

# Dynamic Calibration

## Personalized EAR Normalization

- The app collects the first 15 EAR samples.
- **Initial baseline** `currentMax = max(first 15 raw EAR values)`
- **Fallback baseline:** 0.23
- During runtime, the baseline is updated with expanding max.
- **Normalized EAR** `earNorm = rawEAR / currentMax;`

> Dynamic calibration adapts the system to different users and eye shapes.

# Sampling Rate and Sliding Window

## Matching the Training-Time Temporal Setup

### Timing & Framerate

- The model was trained on EAR sequences sampled around 15 FPS.
- The browser does not run inference on every animation frame.
- **Sampling interval:**
  SAMPLE_INTERVAL_MS = 1000 / 15;

### Window Logic

- **Sliding window size:**
  WINDOW_SIZE = 7;
- Inference only starts after the 7-frame window is full.

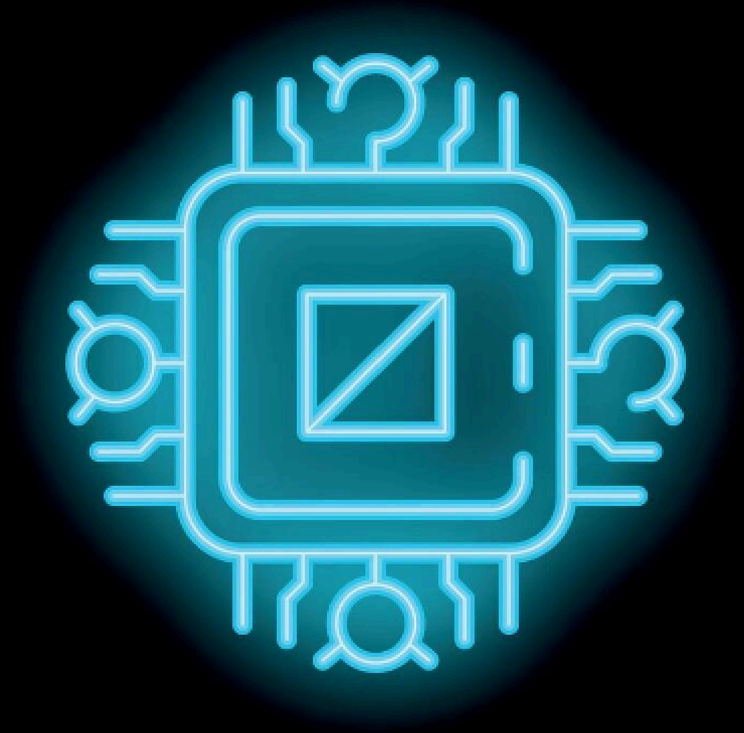
# Hybrid Heuristic Filter

## Reducing Unnecessary Model Calls

- If all normalized EAR values in the window are high, the eye is considered open.

```
if  (Math.min(slidingWindow) >= 0.5 ) {
  prediction = 0;
}
```

- SVM inference only runs when the window looks suspicious.
- **The dashboard shows:**
  - CPU Saved
  - CPU Mode: Filter or SVM



> The hybrid filter reduces CPU usage while preserving real-time responsiveness.

# BlinkGuard Web App Features

## Current BlinkGuard Features

- Start, stop, and reset camera session
- Real-time eye landmark overlay
- Blink count during the session
- Blinks in the last 60 seconds
- 3-minute blink rate trend
- No-blink streak timer
- Drowsy event counter
- SVM prediction display
- CPU saving display
- Notification permission status

# Blink Rate Monitoring

## Detecting Low Blink Rate

### Tracking & Target

- The app stores timestamps for detected blink events.
- Blink rate is calculated from the last 60 seconds.
- **Default target:**
  **12 blinks per minute**

### Warnings

- **Warning levels:**
  - **< 12/min:** Low blink rate
  - **< 7/min:** Very low blink rate

> Use Case:
> Warn users when they focus too long and blink less than expected.

# No-Blink Streak Warning

## Alerting Users Who Forget to Blink

- ⏱ The app tracks time since the last detected blink.
- 🔔 **Warning threshold:**
  LONG_NO_BLINK_MS = 15000;
- ⚠ If the user does not blink for over 15 seconds:
  - Web warning is shown.
  - System notification can be sent.

> This feature targets users who stare at the screen during long focus sessions.



# Drowsiness Warning

Detecting Long Eye Closure

## Class 2

SVM Output

The SVM predicts class 2 for sleep / long closure.

## Smoothing

Temporal Logic

The app uses temporal smoothing:

DROWSY_HISTORY_SIZE = 15;

DROWSY_VOTES_REQUIRED = 10;

## Warning

Trigger Event

If enough recent predictions are class 2, the app triggers a drowsiness warning.

## Dashboard

Counter Update

Drowsy events are counted in the dashboard.

# System Notifications

## Alerts Outside the Web Page

- Uses the Browser Notification API.
- User enables alerts with **Enable Notifications**.
- **Notifications can be sent for:**
  - Long no-blink streak
  - Low blink rate
  - Drowsiness warning
- **Cooldown prevents spam** `NOTIFICATION_COOLDOWN_MS = 60000;`

> Example Notification:
>
> *“You have not blinked for X seconds. Blink a few times and look away from the screen.”*

# Deployment on Cloudflare Pages

## Static Hosting with Cloudflare Pages

- No backend server required.
- No build command required.
- **Output directory:** `deployment/scripts`
- Cloudflare provides HTTPS by default.

  HTTPS is required for webcam and notification APIs outside localhost.

> BlinkGuard can be deployed as a free or low-cost static web app.



# Deployment Summary

## Final Deployment Outcome

- Python SVM model was converted to pure JavaScript.
- MediaPipe JS handles real-time face and eye tracking.
- EAR-based TimeSeries logic runs fully in the browser.
- BlinkGuard monitors blinking behavior and drowsiness.
- System notifications extend alerts beyond the web UI.
- The app is ready for static hosting on Cloudflare Pages.

BlinkGuard demonstrates how a lightweight ML model can become a privacy-friendly, real-time web application without server-side inference.



BlinkGuard Web